

Claude Code

Guida Definitiva · Funzionalità, Comandi, Setup e Workflow

v2.1.83 · 27 Marzo 2026

Terminale & IDE

Voice Mode

1M Context

MCP Tools

Agenti Paralleli

Hooks & Skills

CLAUDE.md

Indice dei Contenuti

1. Cos'è Claude Code e Come Funziona

- Il loop agentic
- Ambienti di utilizzo
- Modello e contesto 2026

2. Installazione & Autenticazione

- Requisiti
- Installazione npm
- Metodi di autenticazione

3. Setup Efficace — Il CLAUDE.md

- Struttura ottimale
- Posizioni dei file
- Inizializzazione automatica

4. Configurazione: settings.json e Variabili

- Gerarchia configurazione
- Chiavi principali
- Variabili d'ambiente

5. Modalità di Permesso e Sicurezza

- Le 5 modalità
- Permessi granulari
- Best practice sicurezza

6. Tutti i Comandi Slash /

- Gestione sessione
- Informazioni e stato
- Configurazione modello
- Memoria e progetto
- Strumenti e integrazioni
- Comandi speciali 2026
- Bundle Skills

7. Shortcut da Tastiera

- Controlli generali
- Cambio modalità
- Prefissi speciali

8. Flags e Opzioni CLI

- Controllo sessione
- Modello e modalità
- Output e debug
- Subcomandi

9. Gestione del Contesto

- Visualizzare con /context
- /compact vs /clear

- Proteggere il contesto

10. Skills, Subagenti e Agent Teams

- Skills con frontmatter
- Posizioni skills
- Subagenti paralleli
- Agent Teams

11. Hooks — Automazione Deterministica

- Tipologie di hook
- Configurazione
- Hook HTTP 2026

12. MCP — Model Context Protocol

- Configurazione
- Scope server MCP
- Server piu usati

13. Metodologia: Come Formulare le Richieste

- Director Mode
- Tecnica Interview
- Annotation Cycle
- Prompting avanzato

14. Il Flusso di Lavoro Ottimale

- Avvio sessione
- Fase Research
- Fase Esecuzione
- Fine sessione handoff

15. Workflow Pratici Completi

- Onboarding repository
- TDD
- Debug sistematico
- Code review
- CI/CD headless

16. Workflow Ottimali Avanzati

- Workflow per team
- Workflow per progetto solo
- Workflow refactoring massiccio
- Workflow documentazione
- Workflow sicurezza

17. File Markdown Ottimali & CLAUDE.md

- Anatomia del CLAUDE.md perfetto
- File .md per category
- Esempi pratici completi
- Come modificare CLAUDE.md
- Template pronti all'uso

1 Cos'è Claude Code e Come Funziona

Claude Code (v2.1.83, marzo 2026) è un agente AI autonomo che opera direttamente nel terminale, nell'IDE o nel browser. A differenza dei chatbot tradizionali, non *suggerisce* codice: **agisce**. Legge file, scrive codice, esegue test, committa su Git e coordina subagenti paralleli, tutto tramite linguaggio naturale.

Il Loop Agentico

Ogni volta che invii una richiesta, Claude Code esegue un ciclo deterministico in cinque fasi:



Ambienti di Utilizzo

Ambiente	Accesso	Note
Terminale CLI	<code>npm install -g @anthropic-ai/claude-code</code>	Principale, full-featured
VS Code / Cursor	Estensione marketplace	Diff inline, @mentions, plan review
Desktop App	<code>claude.ai/download</code>	Fuori dall'IDE, remote control
Browser / Mobile	<code>claude.ai/code</code>	Remote control da qualsiasi device
CI/CD (headless)	<code>claude -p 'task' --dangerously-skip-permissions</code>	Automazione pipeline

Modello e Contesto (Marzo 2026)

Il modello predefinito è **Claude Opus 4.6** (Opus 4 e 4.1 rimossi). Gli abbonati Max, Team ed Enterprise dispongono di una finestra di contesto da **1 milione di token**, sufficiente per interi repository. L'effort di default da marzo 2026 è **medium** (precedentemente high).

■ Il modello fast è configurabile in `modelOverrides.fast` — di default `claude-haiku-4-5`.

2 Installazione & Autenticazione

Requisiti

Requisito	Dettaglio
Node.js	Versione 18 o superiore (LTS raccomandato)
Sistema operativo	macOS, Linux, Windows con WSL2
Account	Claude.ai — Pro, Max, Team o Enterprise (o API key Anthropic)
npm	Incluso con Node.js

Installazione

Installazione

```
# Installazione globale (raccomandato)
$ npm install -g @anthropic-ai/claude-code

# Aggiornamento all'ultima versione
$ claude update

# Via npx (senza installazione permanente)
$ npx @anthropic-ai/claude-code

# Verifica versione installata
$ claude --version # → 2.1.83
```

Metodi di Autenticazione

Autenticazione

```
# Login via browser OAuth (raccomandato per uso personale)
$ claude
$ claude login

# Via API key (ambienti CI/CD, server)
$ export ANTHROPIC_API_KEY=sk-ant-...

# Via Anthropic Console (SSO aziendale)
$ claude --console

# Logout
$ claude logout
```

■ Per VS Code: installa l'estensione 'Claude Code' dal marketplace, poi apri la Command Palette (Cmd+Shift+P) e digita 'Claude Code: Open'. L'estensione sincronizza automaticamente la sessione.

3 Setup Efficace — Il CLAUDE.md

CLAUDE.md è il **cervello persistente** di ogni sessione. Claude lo legge automaticamente all'avvio e sopravvive a /clear e /compact. È qui che definisci convenzioni, comandi di build, architettura e regole che Claude deve seguire sempre. È anche il luogo in cui Claude scrive *auto-memory*: osservazioni sui tuoi pattern di lavoro, raccolte tra sessioni.

Struttura Ottimale CLAUDE.md

CLAUDE.md — Struttura Ottimale

```
# CLAUDE.md — Template Ottimale

## Progetto
Breve descrizione, scopo, stato attuale.

## Stack Tecnologico
- Backend: FastAPI + PostgreSQL 16
- Frontend: React 18 + TypeScript strict
- Test: pytest (min 80% coverage) + vitest

## Comandi Frequenti
- Build: npm run build
- Test: pytest -x --tb=short
- Lint: ruff check . && mypy src/
- Dev: uvicorn main:app --reload

## Convenzioni Codice
- Snake_case Python, camelCase TypeScript
- Conventional Commits: feat/fix/chore/docs
- NEVER modificare .env in produzione
- Prefer X over Y (non solo 'non fare X')

## Architettura
- State: Zustand (vedi src/stores/)
- API: REST v2 (OpenAPI spec in docs/)
- Auth: JWT + refresh token (7 giorni)

## Regole di Sicurezza
- Mai push diretto su main
- Mai rm -rf senza conferma esplicita
```

Posizioni dei File CLAUDE.md

Percorso	Scope	Uso
./CLAUDE.md	Progetto	Condiviso in git con il team

~/.claude/CLAUDE.md	Personale	Caricato in tutti i progetti
/etc/claude-code/	Enterprise	Regole a livello organizzazione (managed)
.claude/rules/*.md	Progetto	Regole per categoria, caricate selettivamente
~/.claude/rules/*.md	Personale	Regole personali per categoria

Inizializzazione Automatica

Nel REPL

```
# Claude analizza il repo e genera CLAUDE.md
$ /init

# Apri e raffina il file generato
$ claude /memory

# Oppure modifica direttamente
$ editor CLAUDE.md
```

■ **CLAUDE.md** viene seguito circa il 70% delle volte. Per regole critiche (mai push su main, mai rm -rf) usa gli Hooks, che sono deterministici al 100%.

■ Regola pratica: se una regola vale per quasi ogni task → CLAUDE.md. Se vale solo per certi workflow → crea una Skill. Non sovraccaricare CLAUDE.md con regole situazionali: consumano token ad ogni sessione.

4 Configurazione: settings.json & Variabili

La configurazione di Claude Code è stratificata: file più specifici sovrascrivono quelli più generici. Capire questa gerarchia è fondamentale per configurare comportamenti diversi per progetti, team e ambienti CI/CD.

Gerarchia dei File di Configurazione

Priorità Configurazione (dal basso verso l'alto)

```
# 1. Managed (Enterprise, sola lettura, priorità più alta)
/etc/claude-code/managed-settings.d/*.json

# 2. Personale (si applica a tutti i progetti)
~/.claude/settings.json

# 3. Progetto (condiviso in git con il team)
.claude/settings.json

# 4. Locale (non committato, preferenze personali)
.claude/settings.local.json

# 5. Stato OAuth, MCP, sessioni
~/.claude.json
```

settings.json — Chiavi Principali

~/.claude/settings.json

```
{
  "model": "claude-opus-4-6-20260301",
  "autoMemoryDirectory": "~/claude/projects/",
  "worktree": { "sparsePaths": ["src/", "tests/"] },
  "modelOverrides": {
    "fast": "claude-haiku-4-5-20251001"
  },
  "attribution": {
    "commit": ""
  },
  "permissions": {
    "allow": ["Bash(git *)", "Read(*)", "Write(src/*)"],
    "deny": ["Bash(rm -rf *)"]
  }
}
```

Variabili d'Ambiente Principali

Variabile	Descrizione	Valori
ANTHROPIC_API_KEY	API key alternativa all'OAuth	sk-ant-...
ANTHROPIC_MODEL	Modello predefinito	claude-opus-4-6-...
CLAUDE_CODE_EFFORT_LEVEL	Effort di default	low / med / high
MAX_THINKING_TOKENS	0 = disabilita thinking esteso	0, 4000, 8000...
CLAUDE_CODE_MAX_OUTPUT_TOKENS	Token massimi di output	32768 (default)
ANTHROPIC_CUSTOM_MODEL_OPTION	Voce custom nel menu /model	Nome modello
CLAUDE_CODE_PLUGIN_SEED_DIR	Dir aggiuntive per plugin	Percorso dir
CLAUDECODE	Nelle shell avviate da Claude Code	= 1
ANTHROPIC_BASE_URL	Proxy o endpoint self-hosted	https://...
IS_DEMO	Nasconde email/org (demo mode)	= 1

5 Modalità di Permesso e Sicurezza

Claude Code chiede conferma prima di ogni azione potenzialmente pericolosa. Le modalità di permesso definiscono quando e come Claude chiede. Il controllo granulare tramite allow/deny nel settings.json permette di definire esattamente quali comandi Claude può eseguire autonomamente.

Le 5 Modalità di Permesso

Modalità	Comportamento	Caso d'uso
default	Chiede conferma per ogni azione rischiosa — bilanciato	Sviluppo quotidiano
acceptEdits	Auto-approva modifiche ai file, chiede per Bash	Refactoring rapido
plan	Read-only: propone ogni azione, attende l'OK	Codebase sconosciute
dontAsk	Nega tutto tranne ciò che è esplicitamente permesso	Produzione controllata
bypassPermissions	Salta tutti i prompt ■ solo ambienti isolati/CI	CI/CD pipeline

Cambio Modalità

Cambio Modalità

```
# Cicla le modalità: Normal → Auto-Accept → Plan → Normal
Shift+Tab

# Avvia direttamente in Plan Mode
$ claude --permission-mode plan

# Bypass totale (■ solo CI/CD isolati)
$ claude --dangerously-skip-permissions
```

Permessi Granulari in settings.json

Permessi Granulari

```
"permissions": {
  "allow": [
    "Bash(git *)",
    "Bash(npm *)",
    "Read(*)",
    "Write(src/*)"
  ],
```

```
"deny": [  
  "Bash(rm -rf *)",  
  "Bash(curl * | bash)"  
]  
}
```

■ Inizia sempre in Plan Mode su codebase sconosciute o prima di refactoring massicci. Revisiona il piano, correggi gli assunti errati, poi sblocca l'esecuzione.

6 Tutti i Comandi Slash /

I comandi slash si digitano nel REPL interattivo preceduti da /. Digita / per vedere la lista filtrata in tempo reale. Oltre ai comandi built-in appaiono anche le tue Skill personalizzate e i comandi MCP connessi.

Gestione Sessione

<code>/clear</code>	Cancella cronologia conversazione (file modificati restano intatti)
<code>/compact [focus]</code>	Compatta il contesto con focus opzionale su cosa mantenere
<code>/resume</code>	Riprende/cambia sessione precedente con UI interattiva
<code>/rename [nome]</code>	Rinomina la sessione corrente
<code>/branch [nome]</code>	Crea un branch della conversazione (alias: /fork)
<code>/rewind</code>	Torna al punto precedente nella conversazione (Esc+Esc in alternativa)
<code>/export [file]</code>	Esporta la conversazione su file o clipboard
<code>/copy [N]</code>	Copia ultima (o N-esima) risposta agli appunti
<code>/diff</code>	Visualizzatore diff interattivo delle modifiche recenti
<code>/add-dir</code>	Aggiunge una directory di lavoro alla sessione

Informazioni e Stato

<code>/cost</code>	Mostra token usati e costo stimato della sessione corrente
<code>/usage</code>	Limiti del piano e stato rate limit attuale
<code>/context</code>	Visualizza uso del contesto come griglia colorata interattiva
<code>/stats</code>	Statistiche sessione e streak di utilizzo
<code>/status</code>	Stato account Anthropic
<code>/doctor</code>	Diagnostica installazione e configurazione
<code>/release-notes</code>	Note di rilascio dell'ultima versione
<code>/help</code>	Lista tutti i comandi disponibili (filtrata mentre digiti)
<code>/insights</code>	Analisi delle sessioni: pattern, costi, frequenze

Configurazione e Modello

<code>/config</code>	Apri interfaccia settings interattiva (Esc=annulla, Enter=salva)
----------------------	--

<code>/model [nome]</code>	Cambia modello (frecche ←→ per cambiare effort)
<code>/effort [livello]</code>	Imposta effort: low / med / high / auto (ultrathink=max)
<code>/fast [on off]</code>	Toggle Fast Mode (stesso modello, output piu veloce)
<code>/vim</code>	Toggle vim keybindings nell'input (modale completo)
<code>/theme</code>	Cambia tema di syntax highlighting
<code>/color [colore]</code>	Imposta colore della barra prompt per la sessione
<code>/permissions</code>	Visualizza e modifica permessi tool
<code>/keybindings</code>	Apre/crea ~/.claude/keybindings.json
<code>/terminal-setup</code>	Configura integrazione terminale (Alt/Option key su macOS)
<code>/privacy-settings</code>	Gestisce preferenze privacy
<code>/output-style</code>	Cambia stile output (Concise, Educational, Code Reviewer...)

Memoria e Progetto

<code>/init</code>	Analizza repo e genera CLAUDE.md di partenza
<code>/memory</code>	Apre CLAUDE.md per modifica (editor o REPL)
<code>/skills</code>	Lista skills disponibili nel progetto e personali
<code>/plan [desc]</code>	Entra in Plan Mode (+ auto-start con descrizione)
<code>/security-review</code>	Analisi sicurezza delle modifiche sul branch corrente
<code>/pr-comments [PR#]</code>	Recupera commenti da una Pull Request GitHub
<code>/install-github-app</code>	Installa integrazione Claude su GitHub Actions
<code>/install-slack-app</code>	Installa integrazione Slack

Strumenti e Integrazioni

<code>/mcp</code>	Gestisce connessioni server MCP (UI interattiva)
<code>/hooks</code>	Gestisce configurazione hook di automazione
<code>/agents</code>	Visualizza e gestisce subagenti in esecuzione
<code>/bashes</code>	Lista e gestisce task in background
<code>/ide</code>	Connette/gestisce integrazione IDE (VS Code, Cursor)
<code>/chrome</code>	Connette Chrome per browser automation
<code>/desktop</code>	Continua la sessione nell'app Desktop

<code>/remote-control (/rc)</code>	Abilita remote control da claude.ai/code
<code>/reload-plugins</code>	Ricarica tutti i plugin a caldo
<code>/plugin</code>	Gestisce il Plugin Marketplace
<code>/sandbox</code>	Esegui in sandbox isolata

Comandi Speciali — Solo 2026

<code>/btw</code>	Domanda laterale senza toccare il contesto — NUOVO marzo 2026
<code>/voice</code>	Push-to-talk vocale (tieni Spazio, rilascia per inviare — 20 lingue)
<code>/loop [intervallo]</code>	Task ricorrente programmato nella sessione
<code>/schedule</code>	Crea task programmati cloud o locali
<code>/effort auto</code>	Reimposta effort al livello predefinito
<code>/remote-env</code>	Gestisce ambienti remoti
<code>/passes</code>	Condividi settimane gratuite con amici

Bundle Skills Built-in (v2.1.63+)

<code>/simplify</code>	Code review parallelo con 3 agenti (sostituisce /review deprecato)
<code>/batch</code>	Modifiche parallele su larga scala (5–30 worktree automatici)
<code>/debug [desc]</code>	Debug sistematico da log di errore
<code>/loop [intervallo]</code>	Task ricorrente programmato
<code>/claude-api</code>	Carica riferimento API e SDK di Anthropic
<code>/btw</code>	Domanda laterale senza costo di contesto

■ /review è deprecato da marzo 2026. Usare /simplify che esegue 3 agenti paralleli.

7 Shortcut da Tastiera

Controlli Generali

Ctrl+C	Cancella input o interrompe generazione corrente
Ctrl+D	Esce dal REPL
Ctrl+L	Pulisce lo schermo
Ctrl+O	Toggle output verbose (vedi tool calls e thinking in tempo reale)
Ctrl+R	Ricerca inversa nella cronologia prompt
Ctrl+G / Ctrl+X Ctrl+E	Apri prompt in editor esterno (\$EDITOR)
Ctrl+B	Mette il task corrente in background
Ctrl+T	Toggle task list persistente
Ctrl+V	Incolla immagine dagli appunti
Ctrl+X Ctrl+K	Termina tutti gli agenti in background — NUOVO
Esc Esc	Rewind: annulla conversazione o solo codice

Cambio Modalita e Modello

Shift+Tab	Cicla: Normal → Auto-Accept → Plan → Normal
Alt+P	Cambia modello a runtime
Alt+T	Toggle thinking esteso on/off (sticky tra sessioni)

Input Testuale

\ + Enter	A capo senza inviare (multilinea rapida)
Ctrl+J	A capo (sequenza di controllo)
Ctrl+K	Cancella dalla posizione cursore a fine riga
Ctrl+U	Cancella intera riga
Ctrl+Y	Incolla ultimo testo cancellato
Alt+B / Alt+F	Sposta cursore di una parola indietro/avanti

Prefissi Speciali

/	Apri lista comandi slash (filtra mentre digiti)
!	Bash mode: esegui comando shell con contesto conversazione
@	File mention: includi file nel contesto con autocompletamento path
/btw	Domanda laterale senza toccare il contesto — NUOVO marzo 2026

■ macOS: i tasti Alt (Alt+P, Alt+T ecc.) richiedono che Option sia configurato come Meta. In iTerm2: Settings → Profiles → Keys → Left Option: Esc+. Esegui `/terminal-setup` per guida.

8 Flags e Opzioni CLI

Controllo Sessione

<code>claude</code>	Avvia REPL interattivo
<code>claude "prompt"</code>	Avvia con prompt iniziale
<code>claude -p 'prompt'</code>	Modalita headless (non-interattivo, per script)
<code>claude -c / --continue</code>	Riprende l'ultima sessione
<code>claude -r 'nome'</code>	Riprende sessione per nome
<code>-n / --name</code>	Imposta nome display della sessione
<code>--new-session</code>	Forza nuova sessione ignorando lo storico
<code>--remote</code>	Sessione web su claude.ai/code

Modello e Modalita

<code>--model / -m</code>	Specifica modello da usare
<code>--effort</code>	Livello di effort: low med high max
<code>--permission-mode</code>	Imposta modalita permesso all'avvio
<code>--dangerously-skip-permissions</code>	Salta tutti i prompt (■ solo CI/CD isolati)
<code>--allowedTools</code>	Pre-approva tool specifici
<code>--bare</code>	Headless minimale (no hooks, no LSP) — NUOVO
<code>--channels</code>	Permission relay via MCP — NUOVO

Output e Debug

<code>--output-format</code> <code>text json stream-json</code>	Formato output (json utile per script)
<code>--json-schema</code>	Output strutturato secondo schema JSON
<code>--verbose / -v</code>	Output verboso di tutte le operazioni
<code>--no-stream</code>	Disabilita streaming (risposta completa alla fine)
<code>--max-turns</code>	Limite turni agentici per sessione
<code>--max-budget-usd</code>	Cap spesa in dollari per la sessione

Contesto e Agenti

<code>--add-dir</code>	Directory di lavoro aggiuntiva
<code>-w / --worktree</code>	Esegui in git worktree isolato
<code>--agent</code>	Usa agente specifico
<code>--mcp-config</code>	File di config MCP JSON
<code>--system-prompt</code>	System prompt aggiuntivo
<code>--append-system-prompt</code>	Appendi al system prompt esistente

Subcomandi CLI

<code>claude update</code>	Aggiorna all'ultima versione
<code>claude mcp add</code>	Aggiunge server MCP alla config
<code>claude mcp list</code>	Lista server MCP configurati
<code>claude mcp serve</code>	Avvia Claude Code come server MCP
<code>claude auth login</code>	Login esplicito
<code>claude auth status</code>	Stato autenticazione corrente
<code>claude config list</code>	Mostra configurazione attiva
<code>claude config set</code>	Imposta parametro di configurazione

9 Gestione del Contesto — Cuore del Workflow

Il contesto è tutto ciò che Claude 'vede' in una sessione: messaggi, file letti, output di comandi, auto-memory. Gestirlo correttamente è la differenza principale tra sessioni produttive e sessioni che vanno a vuoto. **La principale causa di fallimento è il contesto degradato.**

Visualizzare il Contesto con /context

Il comando /context mostra una griglia colorata con l'uso attuale del context window:

```

/context output

CLAUDE.md + System Prompt 12% █████
Autocompact Buffer (riservato) 45% ████████████████████████████████
Conversazione (disponibile) 43% ████████████████████████████████

# Il buffer di autocompact occupa ~45K token dall'avvio.
# Il contesto effettivo disponibile e' quindi inferiore al totale.
# Con 1M window (Max/Team) questo e' meno critico.
```

/compact vs /clear — La Decisione Piu Importante

Aspetto	/compact	/clear
Quando usare	Context >80%, task ancora in corso	Switching a task completamente nuovo
Effetto	Comprime la conversazione in un riassunto	Azzera tutto, CLAUDE.md si ricarica
Perdita info	Si (lossy compression, dettagli dropati)	No (non c'e nulla da perdere)
File modificati	Restano intatti	Restano intatti
Best practice	Max una volta per sessione	Default tra task diversi
Quando NON usare	Come alternativa a /clear	Mid-task con contesto critico

Regola d'Oro: Preferisci /clear a /compact. La compattazione e' una compressione con perdita: dettagli critici vengono scartati e la comprensione dell'intento degrada. Due compattazioni in una sessione sono un segnale che il task era troppo grande o mal strutturato.

Workflow con /clear tra Step

```

Workflow con /clear tra Step

# Strategia ottimale per task lunghi:
```

```
# 1. Scrivi un piano/spec su file
$ cat > plan.md << 'EOF'
## Obiettivo: implementare auth JWT
## Step 1: modello User con hash password
## Step 2: endpoint /login
## Step 3: middleware auth
EOF

# 2. Esegui task per step, poi /clear tra step
> Implementa Step 1 del piano in plan.md
... lavoro ...
$ git commit -m 'feat: user model with password hash'
> /clear

# 3. Prossima sessione con contesto fresco
> Leggi plan.md, implementa Step 2
```

Proteggere il Contesto con i Subagenti

Per task che richiedono di leggere molti file (research, analisi di 50+ file), usa un subagente. Il lavoro pesante avviene in un contesto separato e il tuo thread principale resta pulito.

Delegare al Subagente

```
> "Usa un subagente per analizzare tutti i file in src/legacy/,
poi riportami un riassunto con i pattern principali"
```

10 Skills, Subagenti e Agent Teams

In Claude Code esistono tre livelli di estensione. Capire quando usare ciascuno è fondamentale per un workflow efficiente.

Skills — Workflow Riutilizzabili

Una Skill è un file Markdown con frontmatter che definisce un workflow riutilizzabile. Invocabile come slash command o automaticamente da Claude in base alla descrizione. Dal 2026 Skills e custom commands sono unificati.

Skill con Frontmatter Completo

```
# .claude/skills/security-review/SKILL.md
---
name: security-review
description: Review del codice per vulnerabilita OWASP – usare su ogni PR
allowed-tools: Read, Grep, Glob, Bash(git diff:*)
argument-hint: [file-o-directory]
model: claude-opus-4-6-20260301
effort: high
context: fork
---

Fai una security review su $ARGUMENTS.
Cerca vulnerabilita OWASP Top 10.
Per ogni finding: Severity, Location, Issue, Fix.

# Contesto dinamico (eseguito all'invocazione):
!`git diff HEAD~1 -- $ARGUMENTS`
```

Posizioni Skills

Percorso	Scope	Disponibilita
.claude/skills//	Progetto	Condivise in git con il team
~/.claude/skills//	Personale	Disponibili in tutti i progetti
~/.claude/commands/.md	Personale legacy	Custom commands (ancora supportati)
.claude/commands/.md	Progetto legacy	Custom commands progetto (ancora supportati)

Subagenti — Worker Paralleli

Un subagente è un'istanza Claude separata con contesto proprio, tool list propria e modalità permesso propria. Usa i subagenti per isolare task pesanti senza sporcare il contesto principale.

Subagenti

```
# Lancia subagente dalla CLI
$ claude --agent researcher 'analizza src/legacy/'

# Nel REPL:
> "Spawna un subagente per eseguire i test,
un altro per fare la code review,
riportami i risultati di entrambi"

# Gestisci agenti in esecuzione
> /agents
> Ctrl+X Ctrl+K # termina tutti gli agenti
```

Agent Teams — Agenti che si Coordinano (Sperimentale)

Agent Teams

```
# Abilita (sperimentale)
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1

> "Crea un team: un agente Backend implementa
l'API, un agente Frontend crea i componenti,
un agente QA testa tutto in parallelo"
```

■ *Regola: CLAUDE.md = regole sempre attive. Skill = workflow on-demand. Subagente = task isolato che protegge il contesto. Agent Team = parallelizzazione complessa.*

11 Hooks — Automazione Deterministica

Gli Hooks sono script shell (o HTTP webhook) che si eseguono automaticamente in punti specifici del workflow di Claude. A differenza delle istruzioni in CLAUDE.md (seguite ~70% delle volte), gli Hooks sono **deterministici: scattano sempre**.

Tipologie di Hook

Hook	Quando scatta	Puo bloccare?
PreToolUse	Prima che Claude usi un tool	Si (exit 2)
PostToolUse	Dopo che Claude usa un tool	No (solo feedback)
Stop	Alla fine di ogni turno	Si (puo forzare continuazione)
PostCompact	Dopo la compattazione del contesto	No
Notification	Per notifiche push personalizzate	No
Elicitation	MCP server richiede input strutturato mid-task	NUOVO 2026
ElicitationResult	Intercetta la risposta all'elicitation	NUOVO 2026
CwdChanged	Directory di lavoro cambiata	NUOVO v2.1.83
FileChanged	File modificato rilevato	NUOVO v2.1.83

Configurazione Hook in settings.json

hooks in settings.json

```
"hooks": {
  "PreToolUse": [
    {
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "./scripts/security-check.sh"
      }]
    }
  ],
  "PostToolUse": [
    {
      "matcher": "Write",
      "hooks": [{
```

```
"type": "command",
"command": "ruff format $CLAUDE_FILE_PATH"
}]
}
],
"Stop": [
{
"hooks": [{
"type": "command",
"command": "./scripts/anti-rationalization-gate.sh"
}]
}
]
}
```

Hook HTTP — NUOVO 2026

Gli hook possono ora inviare POST JSON a un URL invece di eseguire script locali. Utile per webhook, notifiche Slack, integrazioni CI/CD.

Hook HTTP

```
"hooks": {
"Stop": [{
"hooks": [{
"type": "http",
"url": "https://hooks.slack.com/services/...",
"method": "POST"
}]
}]
}
```

■ Hook essenziali da configurare subito: (1) PreToolUse su Bash per bloccare `rm -rf` e push su main; (2) PostToolUse su Write per auto-formatting; (3) Stop anti-rationalization per evitare che Claude dichiari vittoria prematura.

12 MCP — Model Context Protocol

MCP (Model Context Protocol) è lo standard aperto di Anthropic per connettere Claude a servizi esterni. Ogni server MCP espone tool che diventano immediatamente disponibili nella sessione come slash commands (es. `/mcp__github__create_pr`).

Configurazione .mcp.json

.mcp.json — Config Progetto

```
# Aggiungi server via CLI
$ claude mcp add github -- npx -y @modelcontextprotocol/server-github

# Oppure in .mcp.json (condiviso in git con il team)
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": { "DATABASE_URL": "${DATABASE_URL}" }
    },
    "remote-api": {
      "type": "url",
      "url": "https://mcp.myservice.com/sse"
    }
  }
}
```

Scope dei Server MCP

Scope	File	Visibilità
Local	.claude/settings.local.json	Solo tu, non in git
Project	.mcp.json	Tutto il team, condiviso in git
User	~/.claude.json	Globale, tutti i tuoi progetti

Server MCP Piu Usati

Server	Package	Funzionalita
GitHub	@mcp/server-github	Issue, PR, repository, code review
PostgreSQL	@mcp/server-postgres	Query SQL direttamente su PostgreSQL
Filesystem	@mcp/server-filesystem	Filesystem con path controllati
Slack	@mcp/server-slack	Messaggi, canali, notifiche Slack
Browser	@mcp/server-browser	Automazione browser / Puppeteer
Memory	@mcp/server-memory	Memoria vettoriale persistente
Jira	@mcp/server-jira	Ticket, sprint, Jira workflow
Figma	@mcp/server-figma	Design tokens, asset Figma
Claude MCP Server	claude mcp serve	Claude Code stesso come server MCP

■ **Attenzione al contesto:** oltre 20K token di MCP tools nell'header riduce drasticamente il contesto disponibile per il lavoro effettivo. Connetti solo i server che usi attivamente nella sessione.

13 Metodologia: Come Formulare le Richieste

Il modo in cui comunichi con Claude Code determina la qualità del risultato più di qualsiasi altra variabile. Esistono pattern che massimizzano l'efficacia.

Director Mode vs Micromanagement

Director Mode

```
# SBAGLIATO – Micromanagement:
> Prima controlla il file auth.py, poi guarda la riga 42,
poi cambia la condizione, poi aggiungi un log...

# CORRETTO – Director Mode con obiettivo chiaro:
> "Implementa autenticazione JWT in Express.js:
- POST /login con validazione email/password
- Middleware auth per rotte protette
- Refresh token con scadenza 7 giorni
- Test con jest per tutti gli endpoint"

# SBAGLIATO – Vago:
> "Migliora il codice"

# CORRETTO – Obiettivo preciso con contesto:
> "Refactoring di src/payments.py per usare il pattern Strategy.
Mantieni 100% compatibilità API esistente.
Lo stack usa Python 3.12 e pytest."
```

Tecnica: Interview prima di Implementare

Per feature complesse, fai fare un'intervista a Claude prima di implementare. Separa sempre la fase di design/planning dalla fase di implementazione.

Interview Workflow

```
> "Voglio implementare [feature]. Intervistami usando
lo strumento AskUserQuestion. Fai domande su:
implementazione tecnica, UI/UX, edge case, tradeoff.
Non fare domande ovvie. Scava nelle parti difficili.
Quando hai tutto, scrivi una spec completa in SPEC.md.
Non implementare ancora."

# Poi inizia una NUOVA sessione con contesto fresco:
> "Leggi SPEC.md e implementa la feature."
```

Tecnica: Annotation Cycle per il Piano

Annotation Cycle

```
# 1. Entra in plan mode e ottieni il piano
> /plan Implementa sistema di pagamenti con Stripe

# 2. Salva il piano su file
> "Scrivi il piano su plan.md"

# 3. Apri plan.md e aggiungi note inline
$ editor plan.md

# Nel file aggiungi ad ogni step sbagliato:
# > NOTE: usa drizzle:generate, non raw SQL
# > NOTE: questo deve essere PATCH, non PUT

# 4. Invia piano annotato con guard phrase
> "Esamina plan.md con le mie note,
correggi tutti i punti. Non implementare ancora."

# 5. Ripeti finche il piano non ha ambiguita
# 6. Solo allora: > 'Implementa il piano'
```

Tecniche di Prompting Avanzate

ultrathink	Attiva effort massimo per il turno corrente (es. 'ultrathink su questo problema')
Alt+T	Toggle thinking esteso on/off (sticky tra sessioni)
use subagents	Lancia piu compute su un problema, contesti separati
/btw	Domanda laterale senza toccare il contesto corrente
grillami su x	Claude fa da reviewer critico dei tuoi ragionamenti
prova senza x	Testa approcci alternativi a quello proposto
sapendo cio, riscrivi	Chiedi la soluzione elegante dopo una fix mediocre

14 Il Flusso di Lavoro Ottimale

I pattern che distinguono gli utenti esperti dai principianti non riguardano i comandi specifici, ma come strutturano l'intero ciclo di lavoro. Tutti i workflow di successo convergono sullo stesso schema fondamentale: **Research → Plan → Execute → Review → Ship**.

1. Avvio Sessione — Sempre Uguale

Avvio Sessione

```
# a) Vai nella directory del progetto
$ cd mio-progetto

# b) Avvia Claude Code
$ claude

# c) Prima volta: inizializza CLAUDE.md
> /init

# d) Controlla stato del contesto
> /context

# e) Opzionale: Plan Mode per task delicati
> Shift+Tab (fino a Plan Mode)
```

2. Fase Research — Prima di Implementare

Non implementare mai senza aver compreso il contesto. Usa `/plan` o la interview technique per esplorare prima di agire.

Research Phase

```
> "Analizza src/auth/ e dimmi: architettura attuale,
problemi che vedi, cosa cambieresti e perche.
Poi proponi un piano per aggiungere 2FA.
Non implementare ancora."

# Oppure usa Plan Mode per vedere cosa farebbe
> /plan Aggiungi 2FA all'autenticazione esistente
```

3. Fase Esecuzione — Task Atomici

Spezza i task grandi. Una sessione = un task completabile. Committa tra step.

Execution Phase

```
> "Implementa il modello User con hash password.
Scrivi i test prima (TDD).
```

```
Committa quando i test sono verdi."

# Claude implementa, testa, committa
# Tu rivedi il diff: Ctrl+O per vedere i tool calls
# Task completato, nuovo task:
> /clear
> "Leggi plan.md, implementa Step 2: endpoint /login"
```

4. Gestione del Contesto durante la Sessione

Context Management durante Sessione

```
# Monitora il contesto
> /cost # token usati
> /context # griglia colorata

# Verde (<50%): tutto ok
# Giallo (50-80%): considera /compact
# Rosso (>80%): /compact ora o /clear e ricomincia

# /compact con focus per mantenere le info chiave
> /compact mantieni i pattern di error handling
e le decisioni architetturali

# Se cambi task: sempre /clear
> /clear
```

5. Fine Sessione — Handoff

Fine Sessione

```
# Committare prima di uscire
> "Committa tutto con conventional commits"

# Scrivi il contesto per la prossima sessione
> "Aggiorna plan.md con lo stato attuale:
cosa abbiamo completato, step successivo,
decisioni prese"

# Controlla costo sessione
> /cost

# Esci
Ctrl+D oppure > /exit
```

■ Il file plan.md (o un documento equivalente) e il vero contesto persistente tra sessioni. E più affidabile della memoria automatica perché è sotto il tuo controllo.

15 Workflow Pratici Completi

Onboarding su Nuovo Repository

Onboarding

```
$ cd nuovo-repo && claude  
> "Analizza questo repository: struttura, stack, dipendenze,  
pattern architetturali, punti deboli, debt tecnico.  
Poi crea un CLAUDE.md completo con le convenzioni che trovi  
e un ARCHITECTURE.md con un diagramma Mermaid."
```

Feature Development con TDD

TDD Workflow

```
> "Implementa TDD per il modulo Cart:  
1. Scrivi test per: aggiungi item, rimuovi, calcola totale con sconti  
2. Fai fallire i test – confermami il fallimento  
3. Implementa il minimo per farli passare  
4. Refactoring se necessario  
5. Committa con: feat(cart): add cart module with discount support  
Non andare al passo successivo senza il mio ok."
```

Debug Sistematico

Debug Workflow

```
> "Debug: TypeError in payments.py riga 47 quando  
amount e None. Output errore:  
[incolla lo stack trace]  
Segui questo processo: RIPRODUCO -> ISOLO -> IDENTIFICO -> FIXO -> VERIFICO.  
Scrivi anche un test che avrebbe catturato questo bug."
```

Code Review Automatico su PR

Code Review

```
$ git diff main..feature/auth | claude -p \  
"Fai code review di questo diff:  
1. Sicurezza (OWASP Top 10)
```

```
2. Performance (N+1 queries, memory leaks)
3. Leggibilità e manutenibilità
4. Test coverage
Dai un voto 1-10 e prioritizza i fix (Critical/High/Medium/Low)"
```

Refactoring con Zero Regressioni

Refactoring Sicuro

```
> "Refactoring di src/api/ per usare async/await ovunque.
PRIMA esegui i test esistenti e nota quanti passano.
Poi refactora in step piccoli, eseguendo i test dopo ogni step.
Fermati se un test rompe e dimmi prima di continuare.
Target: stessa API pubblica, 0 regressioni."
```

Sessione CI/CD Headless

CI/CD Integration

```
# GitHub Actions - analisi e fix automatico
- name: Claude Code Auto-Fix
  run: |
    claude -p 'Analizza i test falliti nel log CI,
    correggi i bug senza cambiare i test,
    committa il fix con il messaggio fix: [descrizione]' \
    --dangerously-skip-permissions \
    --output-format json
```

Documentazione Automatica

Documentazione

```
> "Genera documentazione completa in italiano:
1. README.md con quickstart (< 5 minuti per avviare)
2. docs/API.md con tutti gli endpoint, esempi cURL
3. CONTRIBUTING.md con setup dev e convenzioni
4. docs/ARCHITECTURE.md con diagramma Mermaid
Basa tutto esclusivamente sul codice esistente,
non inventare funzionalità non presenti."
```

Parallelizzazione con /batch

/batch per Cambio Massivo

```
# /batch apre automaticamente N worktree paralleli  
> /batch "Aggiorna tutti i componenti React in src/components/  
da class components a functional components con hooks.  
Un worktree per cartella, max 10 paralleli."
```

16 Workflow Ottimali Avanzati

Questo capitolo raccoglie i workflow più efficaci per scenari specifici e complessi, derivati dalle best practice della community e dall'esperienza d'uso avanzata di Claude Code.

Workflow 1 — Sviluppatore Solo, Progetto Nuovo

Il workflow ottimale quando inizi un progetto da zero come sviluppatore singolo.

Workflow Sviluppatore Solo

```
# === GIORNO 1: SETUP ===  
$ mkdir mio-progetto && cd mio-progetto && git init  
$ claude  
  
# 1. Definisci il progetto con interview  
> "Sono uno sviluppatore solo. Voglio creare [descrizione].  
Intervistami: stack, architettura, requisiti, scope.  
Al termine: genera struttura cartelle, CLAUDE.md,  
README.md bozza, e un piano di sviluppo in plan.md.  
Non scrivere codice ancora."  
  
# 2. Avvia con il primo task atomico  
> /clear  
> "Leggi plan.md. Implementa Step 1: scaffolding base con CI."  
  
# === OGNI SESSIONE SUCCESSIVA ===  
$ claude -c # riprende ultima sessione  
> "Dove eravamo rimasti? Leggi plan.md."  
  
# Al termine di ogni sessione:  
> "Aggiorna plan.md: stato, prossimo step, blocchi."  
$ git commit -m 'feat: ...'  
Ctrl+D
```

Workflow 2 — Team, Feature Branch

Il workflow per team che lavorano su feature branch con review obbligatoria.

Workflow Team - Feature Branch

```
# === FEATURE BRANCH WORKFLOW ===  
  
# 1. Crea il branch  
$ git checkout -b feature/nome-feature  
  
# 2. Leggi la issue / spec  
$ claude
```

```
> "Leggi @docs/spec/feature-xyz.md e @.github/CONTRIBUTING.md.
Proponi piano di implementazione rispettando le convenzioni.
Evidenzia rischi e dipendenze.
Non implementare ancora."

# 3. Implementa con TDD
> /plan Implementa feature-xyz secondo la spec
> Shift+Tab per verificare il piano

# 4. Review e PR
$ git diff main | claude -p "Revisiona il diff per:",
- Conformita alla spec
- Breaking changes
- Test coverage
- Security issues

# 5. Pulisci prima del merge
> "Verifica conventional commits e aggiorna CHANGELOG.md"
```

Workflow 3 — Refactoring Massiccio Sicuro

Come affrontare un refactoring grande (migrazione framework, cambio architettura, upgrade major version) senza regressioni.

Workflow Refactoring Massiccio

```
# === REFACTORING MASSICCIO ===

# FASE 0: Snapshot
$ git stash && npm test -- --coverage > coverage_before.txt

# FASE 1: Analisi (sessione dedicata)
$ claude
> "Analisi completa di src/: pattern usati, dipendenze,
coupling alto, codice duplicato, test mancanti.
Produci REFACTOR_PLAN.md con step ordinati per rischio.
Ordina dal piu sicuro al piu rischioso."

# FASE 2: Esecuzione step by step
> /clear
> "Leggi REFACTOR_PLAN.md. Esegui Step 1 SOLO.
Prima del cambio: esegui test e nota baseline.
Dopo ogni modifica: ri-esegui test.
Fermati se qualcosa si rompe."

# Tra step: commit + clear
$ git commit -m 'refactor(step1): ...'
> /clear
> "Leggi REFACTOR_PLAN.md. Esegui Step 2 SOLO."

# FASE 3: Verifica finale
```

```
$ npm test -- --coverage > coverage_after.txt
$ diff coverage_before.txt coverage_after.txt
```

Workflow 4 — Documentazione Tecnica Automatica

Workflow Documentazione

```
# === DOCUMENTAZIONE AUTOMATICA ===

$ claude
> "Genera documentazione tecnica COMPLETA:

# File da creare:
README.md - Overview + quickstart < 5 min
docs/API.md - Tutti gli endpoint, params, esempi cURL
docs/ARCH.md - Architettura con diagramma Mermaid
CONTRIBUTING.md - Setup dev + convenzioni + PR process
docs/DEPLOYMENT.md - Deploy prod step-by-step

Regole:
- Basa tutto SOLO sul codice esistente
- Usa diagrammi Mermaid per architettura e flussi
- Esempi pratici con comandi copia-incolla
- Lingua: italiano
- Non inventare funzionalita non implementate
- Ogni doc deve essere autonomo"
```

Workflow 5 — Security Audit

Workflow Security Audit

```
# === SECURITY AUDIT ===

$ claude --effort high
> "Esegui security audit completo del repository:

SCOPE: OWASP Top 10 + secrets in codice + dipendenze vulnerabili

Per ogni finding produci:
- Severity: CRITICAL/HIGH/MEDIUM/LOW
- File + linea
- Descrizione vulnerabilita
- Exploit scenario (come potrebbe essere sfruttato)
- Fix raccomandato con codice

Tools: leggi TUTTI i file, usa Bash per npm audit,
cerca pattern hardcoded secrets.

Output: SECURITY_REPORT.md
```

```
Non fixare ancora: prima voglio approvare il report."

# Dopo approvazione, fix per severity:
> "Fixa tutti i CRITICAL in SECURITY_REPORT.md.
Un commit per issue. Titolo: fix(security): CVE-...
Non toccare i Medium e Low."
```

Workflow 6 — Onboarding Collaboratore

Come usare Claude Code per accelerare l'onboarding di un nuovo sviluppatore nel team.

Workflow Onboarding

```
# === ONBOARDING COLLABORATORE ===

# (run dal nuovo collaboratore nella repo del team)
$ claude
> "Sono nuovo in questo team. Analizza il repository e:

1. Spiega l'architettura in modo semplice
2. Identifica i file piu importanti da conoscere
3. Spiega le convenzioni di codice usate
4. Mostra il flusso completo di una request HTTP
5. Identifica le 'gotchas' (cose non ovvie che devo sapere)
6. Dimmi come fare girare il progetto in locale

Produci ONBOARDING.md con tutto questo.
Usa esempi di codice dal repository reale.
Scrivi come se spiegassi a un senior developer."
```

17 File Markdown Ottimali & CLAUDE.md

I file Markdown sono il layer di comunicazione più efficace con Claude Code. Un CLAUDE.md ben costruito e un set di file .md ben organizzati moltiplicano la produttività e la qualità dell'output di ogni sessione.

Anatomia del CLAUDE.md Perfetto

Il CLAUDE.md ottimale bilancia tre esigenze: essere abbastanza specifico da guidare Claude, abbastanza conciso da non sprecare token, e abbastanza flessibile da coprire i casi d'uso reali.

CLAUDE.md Template Completo

```
# CLAUDE.md – Template Completo e Ottimizzato

## Progetto
Nome: MyApp – SaaS per gestione fatturazione B2B
Stato: MVP in sviluppo, 60% completato
Obiettivo attuale: implementare modulo pagamenti Stripe

## Stack Tecnologico
- Backend: Node.js 20 + Express 4 + TypeScript strict
- Database: PostgreSQL 16 + Prisma ORM (schema in prisma/schema.prisma)
- Frontend: Next.js 14 + Tailwind CSS + shadcn/ui
- Auth: NextAuth.js v5 + JWT
- Test: Vitest + Testing Library + Playwright (E2E)
- Deploy: Vercel (frontend) + Railway (backend)

## Comandi Essenziali
- Dev: npm run dev (porta 3000)
- Test: npm test (vitest) | npm run test:e2e (playwright)
- Build: npm run build
- DB: npx prisma migrate dev | npx prisma studio
- Lint: npm run lint (eslint + prettier)

## Convenzioni di Codice
- Conventional Commits OBBLIGATORI: feat/fix/chore/docs/refactor
- TypeScript strict: NO any impliciti, NO assertions non necessarie
- Naming: camelCase variabili/funzioni, PascalCase componenti/classi
- PREFER async/await su Promises chain
- PREFER named exports su default exports
- NEVER modificare i file in prisma/ senza usare npx prisma migrate dev

## Architettura
- API: REST v2 (docs/api/openapi.yaml), versionata in /api/v2/
- Autenticazione: JWT access (15min) + refresh (7gg) in HttpOnly cookie
- State management: Zustand (src/stores/)
- Error handling: ApiError class (src/lib/errors.ts) – SEMPRE usare questa
```

```
- Logging: Pino (src/lib/logger.ts) — NEVER console.log in produzione

## Regole Critiche
- MAI push diretto su main — usa sempre PR
- MAI rm -rf senza conferma esplicita
- MAI modificare .env.production
- SEMPRE eseguire i test prima di committare
- SEMPRE usare transazioni Prisma per operazioni multi-step

## Contesto Auto-Memory
# Claude aggiorna questa sezione automaticamente tra sessioni
```

Come Modificare CLAUDE.md

Ci sono tre modi per modificare CLAUDE.md, ognuno appropriato a diversi scenari.

Come Modificare CLAUDE.md

```
# METODO 1: Via comando /memory nel REPL (piu semplice)
> /memory

# Si apre l'editor configurato in $EDITOR
# Salva e chiudi per applicare le modifiche

# METODO 2: Istruzione a Claude durante la sessione
> "Aggiorna CLAUDE.md: aggiungi Redis come cache layer,
porta default 6379. Aggiungilo alla sezione Stack Tecnologico
e alla sezione Comandi con: redis-cli -p 6379"

# METODO 3: Modifica diretta da terminale
$ editor CLAUDE.md

# Le modifiche sono attive dalla prossima sessione

# BEST PRACTICE: Chiedi a Claude di auto-aggiornare
> "Al termine di questa sessione, aggiorna CLAUDE.md con:
- Le decisioni architetturali prese oggi
- I nuovi pattern di codice introdotti
- Eventuali gotchas che hai scoperto"
```

File .md per Categoria — La Struttura .claude/rules/

Invece di mettere tutto in CLAUDE.md, usa file separati per categoria. Claude li carica selettivamente, risparmiando token nelle sessioni che non ne hanno bisogno.

Struttura .claude/ ottimale

```
.claude/
  ■■■ rules/
  ■ ■■■ testing.md # Regole per scrittura test
  ■ ■■■ security.md # Regole di sicurezza
```

```

■ ■■■ database.md # Convenzioni DB e Prisma
■ ■■■ frontend.md # Regole React/Next.js
■ ■■■ api-design.md # Convenzioni API REST
■ ■■■ git-workflow.md # Processo git e commit
■■■ skills/
■ ■■■ security-review/ # Skill OWASP review
■ ■ ■■■ SKILL.md
■ ■■■ test-generator/ # Genera test da codice
■ ■ ■■■ SKILL.md
■ ■■■ doc-generator/ # Genera documentazione
■ ■■■ SKILL.md
■■■ settings.json # Config progetto condivisa
■■■ settings.local.json # Config personale (gitignore)
■■■ ...
CLAUDE.md # Root: regole globali essenziali

```

Esempio: .claude/rules/testing.md

.claude/rules/testing.md

```

# Regole per la Scrittura di Test

## Framework e Strumenti
- Vitest per unit test (NON Jest)
- Testing Library per componenti React
- Playwright per E2E (directory tests/e2e/)
- MSW per mock delle API nei test

## Struttura dei Test
- Ogni file sorgente ha il suo test: src/lib/auth.ts -> src/lib/auth.test.ts
- Descrizioni in italiano: describe('autenticazione JWT')
- Usa it() non test() per i casi singoli
- Struttura: Arrange -> Act -> Assert (commenta le sezioni)

## Coverage
- Minimo 80% di coverage su src/
- 100% su src/lib/ (logica critica)
- I componenti UI non richiedono coverage minima

## Mock
- PREFER MSW su jest.mock() per le chiamate HTTP
- Mock factory in tests/factories/ (una per entita)
- NEVER mock i moduli interni: refactora invece

## Naming
- Test files: *.test.ts (unit) | *.spec.ts (integration)
- Descrizione: 'dovrebbe [comportamento atteso] quando [condizione]'

```

Esempio: .claude/rules/security.md

.claude/rules/security.md

```
# Regole di Sicurezza

## Gestione Secrets
- NEVER hardcodare chiavi API, password, token nel codice
- Usa SEMPRE variabili d'ambiente da .env.local
- Valida che i secret siano presenti all'avvio (src/lib/env.ts)

## Input Validation
- SEMPRE validare input utente con Zod (src/lib/schemas/)
- Sanitizza prima di inserire in DB (usa Prisma - non raw SQL)
- Rate limiting su TUTTE le route POST/PUT/DELETE

## Autenticazione
- Controlla session PRIMA di ogni operazione privilegiata
- Usa la funzione requireAuth() da src/lib/auth-guard.ts
- NEVER esporre password hash nelle response API

## Headers
- Content-Security-Policy configurato in next.config.js
- CORS solo su domini whitelist (src/lib/cors.ts)
- HTTPS enforced in produzione
```

Template SKILL.md per Skills Personalizzate

.claude/skills/test-generator/SKILL.md

```
---
name: test-generator
description: >
Genera test completi per un file sorgente TypeScript/React.
Usa Vitest + Testing Library. Invoca quando devi scrivere
test per un modulo o componente esistente.
allowed-tools: Read, Write, Bash(npx vitest run *)
argument-hint:
model: claude-opus-4-6-20260301
effort: high
context: fork
---

Genera test completi per il file $ARGUMENTS.

PROCESSO:
1. Leggi il file sorgente e identifica tutte le funzioni/componenti
2. Per ogni unit: scrivi almeno 3 test (happy path, edge case, error)
```

3. Usa le factory in tests/factories/ per i dati di test
4. Scrivi il file di test in [stesso-percorso].test.ts
5. Esegui i test e correggi se necessario
6. Report finale: N test scritti, coverage stimata

REGOLE (da .claude/rules/testing.md):

```
!`cat .claude/rules/testing.md`
```

Esempi Pratici: Costruire un CLAUDE.md da Zero

Tre approcci per costruire il CLAUDE.md di un progetto esistente:

Costruire CLAUDE.md - 3 Approcci

=== APPROCCIO 1: Generazione automatica (repo esistente) ===

```
$ claude
```

```
> /init
```

Claude analizza il repo e genera CLAUDE.md

Poi raffina manualmente le sezioni generate

```
> /memory # apre per editing
```

=== APPROCCIO 2: Intervista guidata (progetto nuovo) ===

```
$ claude
```

```
> "Devo creare il CLAUDE.md per un nuovo progetto.
```

Intervistami su: stack, convenzioni, regole critiche,
comandi di build/test/dev, architettura.

Poi genera il CLAUDE.md completo e ottimizzato.

Includi solo cio che userai effettivamente ogni sessione.

Mantienilo conciso: max 150 righe."

=== APPROCCIO 3: Evoluzione progressiva ===

Inizia con CLAUDE.md minimo:

```
$ cat > CLAUDE.md << 'EOF'
```

```
# CLAUDE.md
```

```
## Stack: Node.js + TypeScript + PostgreSQL
```

```
## Test: npm test
```

```
## Build: npm run build
```

```
EOF
```

Claude aggiunge informazioni durante l'uso

```
> "Aggiorna CLAUDE.md con le convenzioni che hai osservato"
```

File plan.md — Il Contesto Persistente tra Sessioni

Il file plan.md e il complemento piu importante di CLAUDE.md: mentre CLAUDE.md contiene regole sempre valide, plan.md traccia lo stato corrente del lavoro. Claude lo aggiorna automaticamente se istruito.

plan.md — Template

```
# plan.md — Template

## Obiettivo del Progetto
Implementare modulo pagamenti con Stripe (entro 2026-04-15)

## Stato Attuale
Last updated: 2026-03-27
- [x] Step 1: Webhook handler Stripe
- [x] Step 2: Tabelle DB payments e subscriptions
- [ ] Step 3: UI checkout page (in corso)
- [ ] Step 4: Email conferma pagamento
- [ ] Step 5: Dashboard abbonamento utente

## Prossimo Step
Completare checkout page (src/app/checkout/page.tsx)
Componenti necessari: CheckoutForm, PriceCard, OrderSummary

## Decisioni Architetturali Prese
- Pagamento one-time (no subscription per ora)
- Stripe Elements (non Stripe Checkout hosted)
- Webhook su /api/stripe/webhook con idempotency key

## Blocchi e Rischi
- TEST: verificare webhook in locale con stripe CLI
- RISCHIO: gestione 3D Secure per carte europee

## Note per la Prossima Sessione
- La UI di checkout segue il design in /designs/checkout.fig
- Usa il componente Card da shadcn/ui
```

Riepilogo: Gerarchia dei File di Contesto

File	Quando viene letto	Gestito da	Persistenza
CLAUDE.md	Ad ogni avvio sessione	Sviluppatore + Claude	Git, permanente
.claude/rules/*.md	Selettivamente per task	Sviluppatore	Git, permanente
.claude/skills/*.md	Quando skill invocata	Sviluppatore	Git, permanente
plan.md	Ogni sessione (manuale)	Claude + Sviluppatore	Git, aggiornabile
.claude/settings.json	Ad ogni avvio	Sviluppatore	Git, permanente
.claude/settings.local.json	Ad ogni avvio	Sviluppatore	Locale, no git
Auto-memory	Automatico	Claude	~/.claude/projects/

■ Il trio vincente: CLAUDE.md (regole permanenti) + plan.md (stato corrente) + .claude/rules/*.md (regole categorizzate). Insieme formano il sistema di memoria più robusto possibile per Claude Code.

Risorse e Link Utili

Risorsa	URL	Descrizione
Docs ufficiali	code.claude.com/docs	Documentazione ufficiale Claude Code
Changelog	code.claude.com/docs/en/changelog	Changelog aggiornato versione per versione
Cheat Sheet live	cc.storyfox.cz	Cheat Sheet live (v2.1.83)
Best practices	github.com/shanraishan/claude-code-best-practice	Community best practices
Awesome Claude Code	github.com/hesreallyhim/awesome-claude-code	Raccolta risorse, workflow, skills
MCP Registry	github.com/modelcontextprotocol/servers	Server MCP ufficiali e community
Claude.ai	claude.ai/code	Interfaccia web / remote control

Claude Code Guida Definitiva 2026 — Basata sulla Cheat Sheet ufficiale v2.1.83 e aggiornata al 27 Marzo 2026. Questa è una guida non ufficiale prodotta per uso personale. Per informazioni ufficiali e aggiornate consulta sempre code.claude.com/docs.